

Übungsblatt 6
 Vorlesung “Cybersicherheit”
 Sommersemester 2025
 Praktische Übung Hashes

Hash Tables

Ein wichtige Datenstruktur in der Informatik sind die sogenannten Hash-Tables. Sie können u.a. die Suche nach Daten deutlich verkürzen. Hashing ist besonders im Umfeld von dezentralen bzw. verteilten Datenstrukturen relevant. Eine spezielle Form der Hash-Table, ist die Distributed-Hash-Table, die es ermöglicht Daten auf verschiedenen verteilten Knoten (Server oder PCs) zu organisieren. Botnetze greifen unter anderen auf solche Datenstrukturen zurück.

Aufbau einer Hash-Table In Abbildung 1 wird das Grundprinzip einer Hash-Table skizziert. Zu jedem Dateneintrag gibt es einen dazugehörigen Schlüssel (**key**). Anhand des Schlüssels wird jeder Eintrag einem Korb bzw. Behälter (engl. bucket) zugeordnet. Hierbei wird der Schlüssel gehasht und der resultierende Hash-Wert entscheidet über die Zuordnung zum *bucket*.

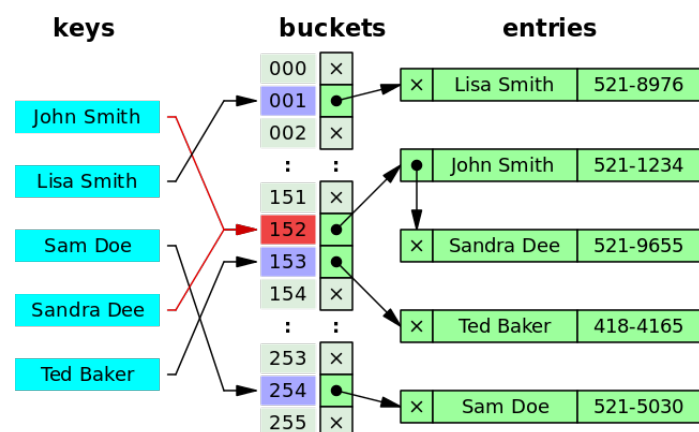


Abbildung 1: Beispiel einer Hash-Table mit *separate chaining* für die Kollisionauflösung
 Bildquelle: <https://de.wikipedia.org/wiki/Hashtabelle>

Übung 1.

Es wird eine einfache Hash-Table mit 6 Buckets angelegt. Als Hashfunktion wird die **Modulo** Operation gewählt. Für 6 Buckets lautet die Hashfunktion für einen key k : $H(k) = k \bmod 6$. Die Buckets entsprechen der Restklassen: $0, 1, 2, 3, 4, 5$. Bei einer Kollision wird nach dem Prinzip aus Abbildung 1 vorgegangen und eine Liste angelegt.

- a) Erstellen Sie eine Tabelle. Jede Spalte repräsentiert eine der 6 Restklassen und fügen Sie nacheinander die Zahlen:
 $71, 36, 22, 38, 11, 10, 1, 6, 4, 112, 42$ ein. Erstellen Sie eine neue Tabelle und fügen Sie die Zahlen:
 $42, 71, 6, 22, 38, 36, 11, 10, 1, 4, 112$ nacheinander ein. Worin unterscheiden sich die beiden Hash-Tabellen?
- b) Es werden nun n Keys auf m buckets verteilt. Die Hashfunktion ist so gewählt, dass sie auf eine Gleichverteilung der Daten abzielt ($P = \frac{1}{m}$).
- Geben Sie die Wahrscheinlichkeit an, dass im Schritt i **keine** Kollision stattfindet.
 - Was ist die Wahrscheinlichkeit dafür, dass es für n Schlüssel **keine** Kollision gibt?
- c) Die Wahrscheinlichkeit, dass **eine** Kollision auftritt kann für N mögliche Hashwerte und für k zufällig eingefügte Werte mit $1 - e^{-\frac{k(k-1)}{2N}}$ approximiert werden. Es sollen $2,3 \cdot 10^7$ Dateien gehasht werden. Wie groß ist die ungefähre Wahrscheinlichkeit bei einem 48-Bit Adressraum, dass eine Kollision auftritt?

Lösung 1.

	0	1	2	3	4	5	0	1	2	3	4	5
a)	36	1	38		22	71	42	1	38		22	71
	6				10	11	6				10	11
	42				4		36				4	
					112						112	

Die beiden Hash-Tabellen unterscheiden sich in der Verkettung der Einträge aufgrund der geänderten Einfügereihenfolge sind die Daten in Spalte '0' in der linken Tabelle anders angeordnet als in der rechten Tabelle.

- b) Um eine allgemeine Formel herzuleiten kann man als ersters die Wahrscheinlichkeit für z.B $m = 6$ betrachten.
1. Es ist noch kein Eintrag vorhanden, daher $P = 1$, dass keine Kollision stattfindet.
 2. Es sind noch 5 Buckets frei, daher ist $P = \frac{5}{6}$.
 3. nur noch 4 freie Buckets, $P = \frac{4}{6}$.
 4. $P = \frac{3}{6}$.
 5. $P = \frac{2}{6}$.
 6. $P = \frac{1}{6}$.
 7. Falls bis jetzt noch **keine Kollision** stattgefunden hat sind **alle 6** Buckets belegt. Folglich muss jetzt eine Kollision stattfinden, $P = 0$.
 Schaut man sich das obige Beispiel an kann man die allgemeine Formel für den Schritt i herleiten:

$$P(i) = \frac{m - (i - 1)}{m}$$

Die Wahrscheinlichkeit, dass es für n Schlüssel **keine** Kollision gibt kann man als Produkt der obigen allgemeinen Gleichung darstellen:

$$\prod_{i=1}^n \frac{m - (i - 1)}{m}$$

c) Wahrscheinlichkeit für **eine** Kollision:

- Bei einem Adressraum von 48 Bit gibt es 2^{48} mögliche Hashwerte $\Rightarrow N$
- $2,3 \cdot 10^7$ mögliche Dateien $\Rightarrow k$

$$1 - e^{-\frac{k(k-1)}{2N}} = 0,609252 \approx 61\%$$

Wobei $k = 2,3 \cdot 10^7$ und $N = 2^{48}$

Hashing of Passwords

Hashing hat eine immense praktische Bedeutung für heutige Web-Apps, die oftmals zusätzliche Dienstleistungen und Funktionen nach dem *Login* bereitstellen. Dies kann zB. eine Zeitung sein, die Zeitungsartikel nur Nutzern mit einem Zeitungsabo bereitstellen. Oder aber auch Beispiele wie Online-Banking, Foren oder digitale Lehre. Die Universität Duisburg-Essen stellt mit der *Moodle* Plattform, eine Möglichkeit sich in Diskussionsforen aktiv mit anderen Studierenden zum Thema der Vorlesung auszutauschen. Es können aber auch über *Moodle* bewertete Abgaben vorgenommen werden. Ein *Login* erfordert die Eingabe eines Benutzernamen und Passworts.

Übung 2.

Es hat sich in der Praxis herausgestellt, dass die Speicherung von Passwörtern im Klartext aufgrund von *Hacks* oder *data leaks* nachteilig ist. Diskutieren und recherchieren Sie folgende Punkte:

- Welche Nachteile bietet das Speichern von Benutzernamen und Passwort im Klartext. Nehmen Sie an ihr Nutzernamen und ihr Passwort ist in einer Datenbank im Klartext gespeichert.
- In der Praxis werden Passwörter gehasht in der Datenbank abgelegt. Welche Vorteile bietet diese Art das Passwort abzuspeichern. Betrachten Sie hierbei die Eigenschaften eines Hashes.
- Auf gehashte Passwörter gibt es einen Angriff namens *Rainbow-Table Attack*, recherchieren Sie und beschreiben Sie den Angriff. Als Gegenmaßnahme, werden die gespeicherten Hashes *gesalzen* (*Salting of Hashes*). Beschreiben Sie das *salting*, warum wird dadurch ein *Rainbow-Table Angriff* verhindert.

Lösung 2.

- Wenn empfindliche Daten in einer Datenbank im Klartext vorliegen kann jeder mit Zugriff diese auslesen.
- Sind die Daten als Hash gespeichert kann aufgrund der Einwegfunktion (Hashing) nicht auf das Klartextpasswort geschlossen werden. Sollten durch einen Hack oder auf andere Weise Daten entwendet werden, so kann der Angreifer mit einem reinen Passworhash unmittelbar noch nichts mit diesen Daten anfangen. Für einen Login benötigt der Angreifer einen Klartext, der aber durch den Hash nicht gegeben ist. Wichtige Eigenschaften vom Hash:
 - Urbildresistenz (Art Einwegfunktion)
Für einen gegebenen Passworhash kann das Klartextpasswort (Eingangswert) nicht berechnet werden.
 - Kollisionsresistenz
Kleinste Änderungen an der Nachricht führen zu einem vollständig neuen Hash.

Der Angreifer kann aufgrund des Hashes nicht auf ähnliche Passwörter schließen.

- c) Ein *Rainbow-Table Angriff* ist eine Passwort-Cracking Methode, die eine spezielle Art von Tabelle (*Rainbow-Table*) benutzt um gehashte Passwörter zu cracken. Eine *Rainbow-Table* ist eine vorberechnete Tabelle mit den Hashwerten zu Klartextpasswörtern. Beim Angriff wird der Passworthash mit der Liste verglichen. Ist dieser vorhanden, so kann das dazugehörige Passwort entnommen werden. Beim *Salting* wird dem Passwort eine zufällig gewählte Zeichenkette vor dem Hashing hinzugefügt. Als *Salt* bezeichnet man die zufällig gewählte Zeichenfolge. Folglich unterscheiden sich die Hashwerte mit und ohne *Salt*. Daher können diese **nicht** mehr in einer *Rainbow-Table* vorberechnet werden.